



Technology Baseline & PoC



Nome del gruppo: **Synergo Unit** - Gruppo 20

Capitolato C6 - Zucchetti

Revisione RTB

Università degli Studi di Padova
Corso di Ingegneria del Software a.a. 2025-2026

Sofia De Blasi, Francesco Marcon, Filippo Idiometri, Armando Moda Scarati,
Anton Ricardo Rupi, Roberto Mariano Doroftei

Obiettivo della Technology Baseline

Obiettivo

Identificare uno stack tecnologico che consenta di:

- sviluppare il Proof of Concept;
- soddisfare i requisiti del prodotto;
- garantire evolvibilità e manutenibilità;
- ridurre i rischi tecnologici principali;
- costituire una base solida per lo sviluppo successivo.



Criteri di valutazione

- compatibilità con i requisiti;
- integrazione con le altre tecnologie dello stack;
- maturità e diffusione della tecnologia;
- qualità della documentazione disponibile;
- curva di apprendimento per il team;
- adeguatezza rispetto all'evoluzione prevista del prodotto.

Requisiti tecnologici di riferimento

Requisiti che hanno guidato la selezione dello stack

Requisito	Impatto
Editing e gestione di documenti Markdown	Frontend
Aggiornamento dinamico dell'interfaccia e anteprima in tempo reale	Frontend
Integrazione con servizi LLM per supporto alla scrittura e analisi dei contenuti	Backend

Requisito	Impatto
Esposizione di API REST per la comunicazione tra componenti	Backend
Persistenza strutturata delle informazioni e dei metadati	Database
Distribuzione uniforme e riproducibile dell'ambiente di esecuzione	Deployment

Framework Frontend



Vue.js

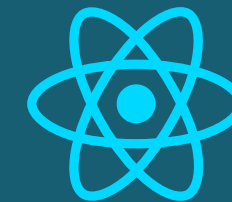
Framework progressivo orientato allo sviluppo di Single Page Application.

Alternative considerate:



Angular

Framework completo e fortemente strutturato.



React

Libreria UI con ecosistema molto ampio e flessibile.

Scelta: Vue.js

Linguaggi:

- TypeScript
- JavaScript (bootstrap applicazione)

Motivazioni:

- Nessuna esperienza significativa del team con framework frontend moderni
- Curva di apprendimento contenuta
- Architettura a componenti adatta all'interfaccia del prodotto da realizzare
- Integrazione semplice con CodeMirror

Motivazioni non scelta:

Angular

complessità non giustificata.

React

maggiore necessità di configurazione architetturale.

Alternative considerate:



CodeMirror

Editor web altamente personalizzabile orientato alla gestione di documenti testuali e Markdown.

Monaco Editor

Editor utilizzato da Visual Studio Code, progettato per scenari assimilabili a IDE.

Scelta: CodeMirror

Motivazioni:

- Supporto nativo al Markdown
- Recupero del testo selezionato
- Modifica programmatica del documento
- Integrazione semplice con Vue.js
- Adeguato alle funzionalità AI del progetto

Motivazioni non scelta:

Monaco Editor

- orientato principalmente a scenari di sviluppo software;
- maggiore complessità e dimensione della libreria;
- funzionalità avanzate non necessarie agli obiettivi del progetto.

Framework Backend



FastAPI

Framework Python per la realizzazione di API REST moderne e asincrone.

Alternative considerate:



Flask

Framework Python minimale e altamente flessibile.



Django

Framework Python completo, con molte funzionalità integrate.

Scelta: FastAPI

Linguaggio: Python

Motivazioni

- supporto nativo alla realizzazione di API REST;
- documentazione OpenAPI generata automaticamente;
- integrazione naturale con l'ecosistema Python;
- integrazione naturale con l'ecosistema AI e LLM.

Motivazioni non scelta:

Flask

- richiede maggiore configurazione manuale;
- minore disponibilità di funzionalità integrate

Django

- completo ma sovradimensionato per il PoC;
- più rigido per un backend usato principalmente come ponte verso LLM.



Python

Linguaggio ampiamente utilizzato nello sviluppo di applicazioni AI e Data Science.

Scelta: Python

Motivazioni

- Ecosistema AI e LLM molto maturo
- Integrazione naturale con FastAPI
- Disponibilità delle librerie richieste
- Presenza di competenze pregresse nel gruppo

Alternative considerate:



PostgreSQL

Database relazionale open source.



MongoDB

Database documentale NoSQL.

Scelta: PostgreSQL

Tecnologia: SQL

Modello: relazionale

Motivazioni

- elevata affidabilità e maturità tecnologica;
- supporto alle proprietà ACID;
- robusta gestione dell'integrità dei dati;
- ampia diffusione in contesti produttivi.

Motivazioni non scelta:

MongoDB

- minori garanzie di consistenza rispetto al modello relazionale;
- flessibilità dello schema non ritenuta necessaria per il dominio applicativo.



Docker Compose
Piattaforma di
containerizzazione e
orchestrazione locale.

Alternative considerate:



Virtual Machine
Ambiente isolato basato su
virtualizzazione completa.

Installazione manuale
Configurazione diretta di
frontend, backend e dipendenze.

Scelta: Docker Compose

Motivazioni

- un unico comando per avviare frontend e backend;
- ambiente riproducibile tra i membri del gruppo;
- minori problemi di versioni locali;
- più leggero di una VM.

Motivazioni non scelta:

Virtual Machine

- maggiore consumo di risorse;
- tempi di avvio superiori;
- isolamento eccessivo rispetto alle necessità.

Installazione manuale

- rischio di configurazioni diverse tra macchine;
- setup più fragile e meno riproducibile.

Obiettivo del Proof of Concept

Obiettivo

Validare la fattibilità tecnica delle principali funzionalità previste per il sistema Second Brain e verificare la corretta integrazione delle tecnologie selezionate nella Technology Baseline.



Perimetro del Proof of Concept

Gestione dei documenti

- editing di documenti Markdown;
- anteprima in tempo reale;
- importazione e salvataggio dei documenti.

Supporto alla scrittura assistita

- applicazione di operazioni di formattazione sul testo selezionato (es. grassetto);
- generazione di suggerimenti tramite la modalità Sei Cappelli per Pensare;
- interazione con il modello LLM mediante prompt;
- visualizzazione e gestione delle proposte generate dall'LLM.



Flusso validato dal PoC

